

Module 3: SQL Data Types

Data types define the kind of data that can be stored in a column. Choosing the correct data type is one of the most important database design decisions because it affects storage, performance, accuracy, and data integrity.



Introduction

Every column in a SQL table must have a data type.

A data type tells SQL Server:

- What kind of data can be stored
- How much storage space is required
- What operations can be performed
- How data should be displayed

Example:

```
CREATE TABLE Employees
(
    EmployeeID INT,
    EmployeeName VARCHAR(100),
    Salary DECIMAL(10,2),
    JoiningDate DATE
);
```

Here:

Column	Data Type
EmployeeID	INT
EmployeeName	VARCHAR
Salary	DECIMAL
JoiningDate	DATE



Learning Objectives

After completing this module, you will be able to:

- ✓ Understand SQL Data Types

- ✓ Differentiate between numeric and character data
- ✓ Store dates and times correctly
- ✓ Select appropriate data types
- ✓ Optimize database storage
- ✓ Avoid common design mistakes



Table of Contents

1. What is a Data Type?
2. Why Data Types Matter
3. Categories of Data Types
4. Numeric Data Types
5. Character Data Types
6. Date and Time Data Types
7. Boolean Data Types
8. Binary Data Types
9. Choosing the Right Data Type
10. Best Practices
11. Summary
12. Practice Questions

1 What is a Data Type?

A Data Type defines the type of value that can be stored in a column.

Think of a data type as a container.

Example:

Age

can only store numbers.

Name

can store text.

JoiningDate

can store dates.

Example

```
CREATE TABLE Students
(
  StudentID INT,
  StudentName VARCHAR(100),
  Age INT
);
```

2 Why Data Types Matter

Selecting the correct data type provides:

Better Performance

Smaller storage = Faster queries

Data Integrity

Prevents invalid values.

Example:

Age INT

Cannot store:

Twenty

Storage Optimization

Efficient use of disk space.

Easier Data Analysis

Proper calculations and sorting.

3 Categories of SQL Data Types

SQL Server Data Types can be divided into:

```
Data Types
|
|— Numeric
|— Character
|— Date & Time
|— Boolean
|— Binary
|— Special Types
```

4 Numeric Data Types

Used to store numbers.

INT

Stores whole numbers.

Range:

```
-2,147,483,648
to
2,147,483,647
```

Storage:

```
4 Bytes
```

Example:

```
Age INT
```

Valid Values:

```
18  
25  
100
```

SMALLINT

Smaller integer values.

Range:

```
-32,768  
to  
32,767
```

Storage:

```
2 Bytes
```

Example:

```
Year SMALLINT
```

BIGINT

Large integer values.

Range:

```
-9 Quintillion  
to  
+9 Quintillion
```

Storage:

8 Bytes

Example:

Population **BIGINT**

TINYINT

Stores small positive integers.

Range:

0 to 255

Storage:

1 Byte

Example:

Rating **TINYINT**

DECIMAL

Stores exact decimal numbers.

Syntax:

DECIMAL(p,s)

Where:

p = Precision
s = Scale

Precision

Total digits.

Scale

Digits after decimal point.

Example:

```
Salary DECIMAL(10,2)
```

Meaning:

```
12345678.90
```

Maximum:

10 digits total

2 digits after decimal

NUMERIC

Same as DECIMAL.

```
Salary NUMERIC(10,2)
```

FLOAT

Stores approximate decimal values.

Example:

```
InterestRate FLOAT
```

Use For:

- Scientific calculations
- Statistical calculations

Avoid for money calculations.

MONEY

Stores currency values.

Example:

```
Salary MONEY
```

Numeric Data Type Comparison

Data Type	Storage	Example
TINYINT	1 Byte	100
SMALLINT	2 Bytes	30000
INT	4 Bytes	500000
BIGINT	8 Bytes	9999999999
DECIMAL	Variable	45000.50
FLOAT	Variable	3.141592
MONEY	8 Bytes	₹50000.25

5 Character Data Types

Used for storing text.

CHAR

Fixed-length character data.

Example:

Gender **CHAR(1)**

Stored:

M
F

If stored value:

A

SQL still reserves:

1 Character

VARCHAR

Variable-length character data.

Example:

Name **VARCHAR(100)**

Storage depends on actual text.

Example:

Kunal

Uses fewer bytes than:

Kunal Kumar Yadav

CHAR vs VARCHAR

Feature	CHAR	VARCHAR
Length	Fixed	Variable
Storage	Fixed	Flexible
Performance	Slightly Faster	Efficient Storage

NCHAR

Fixed-length Unicode data.

Supports:

- Hindi
- Chinese
- Japanese
- Arabic

Example:

```
Name NCHAR(50)
```

NVARCHAR

Variable-length Unicode data.

Most commonly used for multilingual systems.

Example:

```
Name NVARCHAR(100)
```

TEXT

Stores very large text.

Example:

Description TEXT

Modern SQL Server prefers:

VARCHAR(MAX)

Character Data Type Comparison

Type	Unicode	Fixed
CHAR	No	Yes
VARCHAR	No	No
NCHAR	Yes	Yes
NVARCHAR	Yes	No

Date and Time Data Types

Used for storing dates and times.

DATE

Stores only date.

Example:

JoiningDate **DATE**

Stored:

2026-06-01

TIME

Stores only time.

Example:

```
LoginTime TIME
```

Stored:

```
10:45:20
```

DATETIME

Stores both date and time.

Example:

```
CreatedAt DATETIME
```

Stored:

```
2026-06-01 10:45:20
```

DATETIME2

Improved version of DATETIME.

Advantages:

- ✓ Larger date range
- ✓ Better precision
- ✓ Recommended by Microsoft

Example:

```
CreatedAt DATETIME2
```

SMALLDATETIME

Less storage.

Stores:

```
Date + Time
```

with reduced precision.

Date Data Type Comparison

Type	Date	Time
DATE	Yes	No
TIME	No	Yes
DATETIME	Yes	Yes
DATETIME2	Yes	Yes
SMALLDATETIME	Yes	Yes

7 Boolean Data Type

SQL Server does not have TRUE/FALSE.

Instead:

```
BIT
```

is used.

Example:

```
IsActive BIT
```

Values:

```
1 = True  
0 = False
```

Example Table

EmployeeID	IsActive
101	1
102	0

Binary Data Types

Used for storing files.

Examples:

- Images
- PDFs
- Videos

BINARY

Fixed-length binary data.

VARBINARY

Variable-length binary data.

Example:

```
ProfilePicture VARBINARY(MAX)
```

Common Uses

Store:

Images
Documents
Audio Files

Choosing the Right Data Type

Employee ID



EmployeeID **INT**



EmployeeID **VARCHAR(50)**

Salary



Salary **DECIMAL(10,2)**



Salary **FLOAT**

Employee Name



EmployeeName **VARCHAR(100)**

Joining Date



JoiningDate DATE

Active Status



IsActive BIT

10 Best Practices

Use Smallest Appropriate Type

Good:

Age TINYINT

Bad:

Age BIGINT

Avoid FLOAT for Currency

Good:

Salary DECIMAL(10,2)

Bad:

Salary **FLOAT**

Use VARCHAR Instead of CHAR

Unless length is fixed.

Use DATETIME2

Instead of DATETIME.

Use NVARCHAR for International Applications

Supports all languages.

Avoid Deprecated Types

Avoid:

TEXT
NTEXT
IMAGE

Prefer:

VARCHAR(MAX)
NVARCHAR(MAX)
VARBINARY(MAX)

Example Table Using Best Practices

```
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,
    EmployeeName NVARCHAR(100),
    Salary DECIMAL(10,2),
    JoiningDate DATE,
    LastLogin DATETIME2,
    IsActive BIT
);
```



Summary

In this module, you learned:

- ✓ What Data Types are
 - ✓ Numeric Data Types
 - ✓ Character Data Types
 - ✓ Date & Time Data Types
 - ✓ Boolean Data Types
 - ✓ Binary Data Types
 - ✓ Data Type Selection
 - ✓ Storage Optimization
 - ✓ Best Practices
-



Practice Questions

Theory

1. What is a Data Type?
2. Difference between CHAR and VARCHAR?
3. Difference between DATE and DATETIME?
4. What is Precision?
5. What is Scale?
6. Difference between INT and BIGINT?
7. Why avoid FLOAT for salary?
8. What is NVARCHAR?
9. What is BIT?
10. What is VARBINARY(MAX)?

Practical

Task 1

Create a table:

Employees

Columns:

EmployeeID
EmployeeName
Salary
JoiningDate
IsActive

Use appropriate data types.

Task 2

Insert 10 records.

Task 3

Display all records.

Task 4

Create a table:

Products

with:

ProductID
ProductName
Price
ManufactureDate
Description

Choose the best data types.



Next Module

→ Module 4: CREATE TABLE Statement

Topics Covered:

- CREATE TABLE Syntax
- Column Definitions
- Constraints
- Naming Conventions
- Real-World Table Design
- Best Practices